Weekly meetings

What is the problem? Measure selection

We are given a training set $D = \{(x_i, y_i)\}_{i=1}^n$ and an index $j \in [n]$ of a data point in the training set. We are also given a learning algorithm $\mathcal{A}(D)$, which, for simplicity, we will assume to be gradient flow on the empirical risk. Our goal is to find a modification function $\mathcal{M}$ for the algorithm $\mathcal{A}$ that will approximate the effect of retraining the model from scratch on $D \setminus (x_j, y_j)$. To do so, we want to choose $\varepsilon(t)$ from **??** among a class of functions E_s that will allow us to forget the j-th data point as much as possible before time T, while achieving a good performance as a predictor.

For this problem to be well posed, we first need a proper definition for "forgetting a data point". In the literature, I could find two different definitions for this concept, which are the following:

- *Certified Removal* [6]: given $\alpha > 0$, we say that $\mathcal{M}$ is an α-certified removal algorithm if:

$$e^{-\alpha} \leq \frac{\mathbb{P}(\mathcal{M}(\mathcal{A}(D), j) \in S)}{\mathbb{P}(\mathcal{A}(D \setminus \{(x_j, y_j)\}) \in S)} \leq e^{\alpha} \quad \forall S \subseteq \Theta.$$

- Same distribution [3]: let $\mathbb{D}_{\mathcal{M}}$ denote the distribution of models learned using mechanism $\mathcal{M}$ on D trying to unlearn the j-th data point. Let $\mathbb{D}_{\text{real}}$ be the distribution of models learned using $\mathcal{A}$ on $D \setminus \{(x_j, y_j)\}$. $\mathcal{M}$ is a good unlearning algorithm if $\mathbb{D}_{\mathcal{M}} = \mathbb{D}_{\text{real}}$. This definition is equivalent to 0-CR.

However, I don't think either of these definitions is what I am looking for. In fact, I am convinced that as long as $\mathcal{M}(\mathcal{A}(D), j)$ ends in a point that can be reached by $\mathcal{A}(D \setminus \{(x_j, y_j)\})$, then we can say that $\mathcal{M}$ is a good unlearning algorithm. The reason for this is that the gradients only depend on the spatial position of θ and not on time. Thus, I'm expecting that there is no way to extract information about the forgotten data point from a gradient that can be computed without using such data.
The main advantage of this definition is that if $\mathcal{M}$ was the oracle algorithm, it would be considered a good unlearning algorithm, whereas for the previous definitions it is not.

Written more formally, I would like to say that $\mathcal{M}$ is a good unlearning algorithm if $\mathbb{P}(\mathcal{M}(\mathcal{A}(D), j) \in B_\beta(\mathcal{A}(D \setminus \{(x_j, y_j)\}))) = 1$ for any $\beta > 0$, where $B_\beta(\mathcal{A}(D \setminus \{(x_j, y_j)\}))$ is the ball of radius β around $\mathcal{A}(D \setminus \{(x_j, y_j)\})$. Note that if the algorithm $\mathcal{A}$ starts from a random initial condition, then $\mathcal{A}(D)$ is a random variable.
For the sake of clarity, I will rewrite this condition in the notation we have been adopting so far.

Definition 1. We say that $\varepsilon(t)$ induces a good unlearning algorithm $\mathcal{M}$ (i.e., $\mathcal{M}(\theta) = \xi(0, T, \varepsilon(t); \theta)$) if, for each $\theta_0 \in \Theta_0 \subseteq \Theta$, for each $\beta > 0$ there exist (at least) a feasible initial condition $\theta \in \Theta_0$ and $t > 0$ such that:

$$\mathcal{M}(\theta_0) \in B_\beta(\xi(0, t, \mathbf{0}; \theta)).$$

This definition makes sense because by the "continuous dependency from initial conditions" theorem[1], the flux of those two differential equations will stay arbitrarily close from T on.
Additionally, this theorem gives us an explicit bound on the distance between the two trajectories after a time t_1 from the meeting point, if they have minimum distance β:

$$\|\xi(0,\cdot,\mathbf{0};\xi(0,T,\varepsilon(t);\theta_0)) - \xi(0,\cdot,\mathbf{0};\xi(0,t,\mathbf{0};\theta))\|_{\infty,[0,t_1]} \leq e^{Lt_1}\beta,$$

where L is the Lipschitz constant of the $-\nabla_\theta R_j$.

Lemma 0.0.1. *Call $\tilde{\theta}(s) = \xi(0,s,\mathbf{0};\xi(0,T,\varepsilon(t);\theta_0))$ and $\theta(s) = \xi(0,s,\mathbf{0};\xi(0,t,\mathbf{0};\theta))$. Assuming that the data points are normalized, i.e. $\|x_i\|_2 = 1$ for $i = 1,\dots,n$, it holds true that for any $s \in [0,t_1]$:*

$$\|\nabla L(\tilde{\theta}(s)) - \nabla L(\theta(s))\|_2 \leq ne^{Lt_1}\beta.$$

Proof. Assume $\Theta \subseteq \mathbb{R}^d$ with the Euclidean norm. Let's compute the difference between the two gradients:

$$\begin{aligned}\|\nabla L(\tilde{\theta}(s)) - \nabla L(\theta(s))\|_2 &= \|\sum_{i=1}^n x_i(x_i^\top\tilde{\theta}(s) - y_i) - \sum_{i=1}^n x_i(x_i^\top\theta(s) - y_i)\|_2 = \\ &= \|\sum_{i=1}^n x_i x_i^\top(\tilde{\theta}(s) - \theta(s))\|_2 \leq \sum_{i=1}^n \|x_i\|_2^2\|\tilde{\theta}(s) - \theta(s)\|_2 \leq ne^{Lt_1}\beta.\end{aligned}$$

■

Remark. If we can approximate exactly $\theta^\star_{-j}$, then all the definitions are satisfied. Namely, in the case of Linear Regression with the squared loss, the Newton method (IF estimation) is exact and therefore it's a good unlearning algorithm.

Some applications of the influence functions I would eventually like to investigate include:

- recognition of informative images in a set;
- variation of weights for mislabeled samples (in this case, it is useful to study what is the $\varepsilon(t)$ that minimizes the distance from optimum at the end of the training).

Critique and new proposal

The previous definition I proposed doesn't actually take into consideration the probability of our end point being close to a feasible trajectory. Therefore, if the set of initial conditions is the whole space, the former definition is empty.
To address this problem, I would like to modify the definition, taking into account the following family of probablity measures.

[1] see: `https://poisson.phc.dm.unipi.it/~sgubin/study_res/year2/anal2.pdf` for the proof

Let $(\mathbb{P}_t)_{t\in[0,+\infty]}$ be a family of probabilty measures on Θ and define $\text{supp}\mathbb{P}_0 = \Theta_0 \subseteq \Theta$ as the set of possible initial conditions. Each probability $\mathbb{P}_t$ is then defined as:

$$\mathbb{P}_t[B] = \mathbb{P}_0[\xi_t^{-1}(B)] \quad \forall B \in \mathcal{B}(\Theta),$$

where $\xi_t(\theta_0) = \xi(0, t, \mathbf{0}; \theta_0)$ is the gradient flow at time t and $\mathcal{B}(\Theta)$ is the Borel sigma algebra over Θ (i.e., it's the push-forward of $\mathbb{P}_0$ by LOO flow). Note that, since $\xi_t : \Theta \to \Theta$ is continuous, $\xi_t^{-1}(B) \in \mathcal{B}(\Theta)$ for any $B \in \mathcal{B}(\Theta)$ and thus $\mathbb{P}_t$ is well defined.

Remark. From the definition of $\mathbb{P}_t$ we can observe that $\text{supp}\mathbb{P}_\infty = \{\theta \in \Theta \mid \nabla_\theta R_j = 0\}$, i.e., $\mathbb{P}_\infty$ is supported only on the set of critical points for the LOO loss function.
In my master thesis, I addressed such set as the "overfitting region".

Definition 2. We say that $\varepsilon(t)$ induces an (α, δ)-good unlearning algorithm at time s if, for all $\theta_0 \in \Theta_0$:

$$\frac{\mathbb{P}_s[B_\delta(\xi(0, s, \varepsilon; \theta_0))]}{\mathbb{P}_s[B_\delta(\xi(0, s, \mathbf{0}; \theta_0))]} \geq 1 - \alpha.$$

To be coherent with Definition 1, we should impose $s = T$.
Let's now see what this definition implies for extreme values of the parameters:

- if $s = 0$, then every algorithm is a good unlearning algorithm (it makes sense since in both LOO and modification we can start from any point in Θ_0 so we don't need to modify anything);
- if $s = +\infty$, then also every algorithm works, since $\varepsilon \in E_s$ and therefore it is constantly 0 after T.
- for $\delta \to \infty$ then $\alpha \to 0$, and, for $\delta \to 0$, $\alpha \to \mathbb{1}_{\xi_s(\Theta_0)}[\xi(0, s, \varepsilon; \theta_0)]$ (not sure).
- if $\Theta_0 = \{\theta_0\}$, call $\bar{\delta} = \sup_{t>0} \|\xi(0, t, \varepsilon; \theta_0) - \xi(0, t, \mathbf{0}; \theta_0)\|$. Then we can express $\alpha(\delta)$ explicitly as $\alpha = \mathbb{1}_{[\bar{\delta},+\infty]}(\delta)$;

To make the definition even less strict, we may take the mean value over θ_0 instead of requiring the condition to hold for each $\theta_0 \in \Theta_0$.

Figure 1 reports a visual representation of the previous definition. The idea here is that we want the attracting regions to be the same for both the learning dynamics.
Namely, if we take two gradient flows initialized on the same point relative to two different datasets, the further the training goes, the more distinguishable the distribution of parameters become. In fact, the estimators generated by the neural nets learn the characteristics of the data in a sequential way [13], causing the observational noise to be learnt at the end of training. This means that the longer the training goes, the more specialised the model becomes, leading to its parameters to converge further away from each others.
As a side note, note that δ carries the meaning of time incertitude, as ξ is also continuous in time.

From Figure 2, I think there is something wring with definition 2. In fact, it looks like it makes more sense to compare two different measures of the same set, rather than

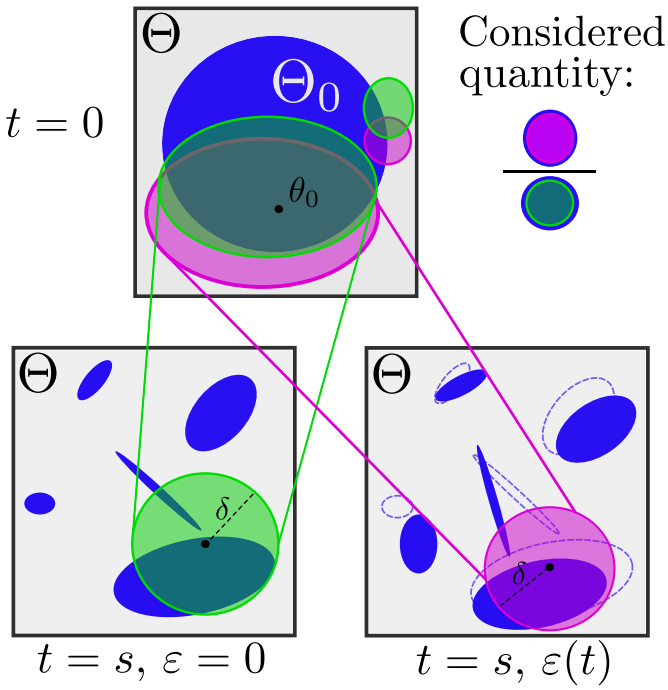

Figure 1: Example representation of the meaning of Definition 2 when $\Theta_0 \subset \mathbb{R}^2$.

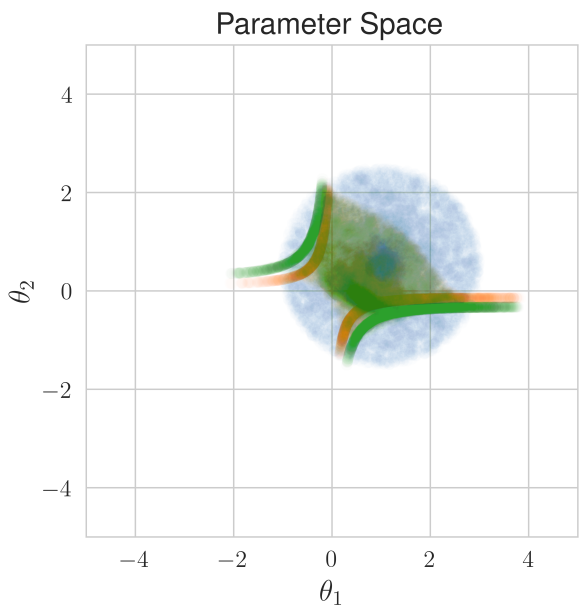

Figure 2: Experiments depicting Definition 2 when $\Theta_0 \subset \mathbb{R}^2$. Blue points represent the 10^4 initial conditions; green points and orange points respectively the LOO retraining and the influence function unlearning after 100 iterations.

two sets with the same measure. In other words, taken the δ-ball around the end point of the modified algorithm, it may be better to consider the ratio between its $\mathbb{P}_t$ measure and $\mathbb{P}'_t$ measure, where $\mathbb{P}'_t$ is the push-forward measure by the modified algorithm.
In this way, we also fix another problem: in the previous case, applying a permutation of neuron parameters before retraining would lead to a bad unlearning algorithm, while in practice it is the same as LOO.

Definition 3. Given two training algorithms $\mathcal{A}$ and $\mathcal{A}'$, define the δ-distinguishability at time s as:

$$\Delta_s^\delta(\mathcal{A}, \mathcal{A}'; \theta_0) = \frac{\mathbb{P}_s[B_\delta(\mathcal{A}'(s, \theta_0))]}{\mathbb{P}'_s[B_\delta(\mathcal{A}'(s, \theta_0))]},$$

where $\mathbb{P}_s$ is the push-forward measure by $\mathcal{A}(s, \cdot)$ and $\mathbb{P}'_s$ is the push-forward measure by $\mathcal{A}'(s, \cdot)$.

Definition 4. Given two training algorithms $\mathcal{A}$ and $\mathcal{A}'$, we say that they are (α, δ)-indistinguishable at time s if:

$$\exp(-\alpha) \leq \mathbb{E}_{\theta_0 \sim \mathbb{P}_0} \left[\Delta_s^\delta(\mathcal{A}, \mathcal{A}'; \theta_0)\right] \leq \exp(\alpha).$$

Remark. If, in Definition 3, we measure the set $B_\delta(\theta_0)$ instead, then the notion of (α, δ)-indistinguishability is very close do ϵ-certified unlearning! The differences are:

- We assume that the only source of randomness is the initialization;
- We take the expectation over θ_0 instead of checking for all the borel sets;
- If we ask the condition for all θ_0, it's still different, since we fixed δ.

Overall, I think these features are quite marginal and I don't have any reason to prefer our version to theirs.

Idea: the longer the training goes, the more the distributions $\mathbb{P}_t$ and $\mathbb{P}'_t$ become specialized, meaning that they concentrate around dynamically important regions.
For this reason, it may be still a good idea to consider the next definition: maybe we can give a lower bound to that quantity, saying that *"given $\delta > 0$, either $\Delta_T^\delta(\mathcal{A}, \mathcal{A}'; \theta_0) = 0$, or it is greater than $\beta(T, \delta)$"*.

Definition 5. Given a training algorithm $\mathcal{A}_D$ on the dataset D and a minimum number of training steps T, an unlearning algorithm $\mathcal{M}$ is valid if for any $\delta > 0$ it holds:

$$\Delta_T^\delta(\mathcal{A}_{D \setminus \{z_j\}}, \mathcal{M}(\mathcal{A}_D, \{z_j\}); \theta_0) > 0 \quad \forall \theta_0 \in \Theta_0.$$

MIA and overfitting

From the previous discussion, it seems like algorithms that overfit are bad when tested with the new measure.
Does it makes sense from a privacy point of view?

The answer is yes. In fact, let's consider the following very naive example of Membership Inference Attack: in a white-box setting (the attacker has full access to the model), if our algorithm completely overfits, it means that it will have 0 loss on the training set. Therefore, the attacker can check if a point z has been used in the training by just checking if the loss of the model on z is 0 or not.

Even in the less extreme setting of [1], the quantity that the authors consider (pp.8,9) increases if we choose an overfitting algorithm. On the other hand, if we could have an oracle procedure that could give us the generating function of the dataset (to which observational noise is added), that algorithm would be independent of initial dataset and therefore be a private algorithm. However, this concept may be more similar to differential privacy rather than unlearning.

Indeed, if $\mathcal{A}$ is such an algorithm, we could write $\mathcal{A}(\mathcal{D}) = \mathcal{A}(\mathcal{D}')$ for any two dataset whose elements are drawn from the same distribution. This means that $\mathcal{A}$ is a 0-DP algorithm.

Maybe, to address this problem, could be interesting to use the Vicinal Risk Minimization algorithm [4], instead of the ERM one, as it looks to me that may get closer to the optimal distribution rather than the overfitting one.
The reason for that is that with VRM we are minimizing a quantity that has a continuous distribution instead of discrete: the ERM can be written in the form of

$$\min_{\theta\in\Theta} \int_{\mathcal{X}\times\mathcal{Y}} (f(x;\theta) - y)^2 d\,\sharp_{\mathcal{D}}(x,y) = \min_{\theta\in\Theta} \sum_{i=1}^{n} (f(x_i;\theta) - y_i)^2,$$

where $\sharp_{\mathcal{D}}$ is the empirical distribution of the dataset $\mathcal{D}$. On the other hand, the idea behind VRM is to extend the empirical distribution by choosing a continuous probability for x, while keeping y constant, i.e.:

$$\min_{\theta\in\Theta} \sum_{i=1}^{n} \int_{\mathcal{X}} (f(x;\theta) - y_i)^2 d\,P_i(x),$$

with P_i being a probability distribution that is concentrated around x_i (for example, the normal distribution centered in x_i: $P_i = \mathcal{N}_\sigma(x - x_i)$. Computations in this case lead to the L_2 regularization!).

Observations on fairness definition: The previous framework allows us to model a larger class of risks and I think it's suited particularily well for our case study:

Definition 0.0.1. Given a probability distribution ρ on $\mathcal{Z} = \mathcal{X} \times \mathcal{Y}$ and a loss function $\ell : \mathcal{Z} \times \Theta \to \mathbb{R}$, I define the risk as:

$$R(\rho, \theta) = \mathbb{E}_{z\sim\rho}[\ell(z, \theta)].$$

With this definition, we can model the theoretical risk as $R(\mathcal{D}, \theta)$ where $\mathcal{D}$ is the true data distribution and the empirical risk as $R(\sum_{i=1}^{n} \delta_{x_i}\delta_{y_i}, \theta)$.

With this setting in mind, the fairness definition conveys a risk that is represented by the measure $\rho_{\text{fair}} = \frac{1}{|S|} \sum P_s$ where $(x_{s_i}, y_{s_i}) \sim P_s$ for all $s \in S$.

Observe that this definition tells us that there exist two types of information leaks (overfitting): one that is data-related and one that is distribution-related! In fact, overfitting is the phenomenon where the model achieves low training score but high generalization error; this can happen both when we use the empirical risk to train and when we use ρ_{fair}. I think that knowing each P_s, we can create a test to check if data is coming from user s or not just by knowing ρ_{fair}.

Interesting point: the previous attack is not the same as the MIA where you check for overfitting evaluating the loss on a specific point.

Idea: the sequential learning described above may be restated in a distributional way: at first, we start from a generic distribution over $\mathcal{Z}$, then we specialize on the different P_s's and finally the distribution dynamics converges to the sum of delta functions on the training data.

Question: Let's go back to the first article we reviewed. What does it mean for the unlearning that we are approximating PBRF with IFs?
Answer: It means that if we use the PBOM (Proximal Bregman Objective Minimization) algorithm, then the influence functions approximate well the unlearning. Maybe we can analyze with our definitions how good the unlearning is and how different is the accuracy compared to ERM training.

Algorithm implementations: D2D vs R2D

An application for which we may want to use influence functions is unlearning a data point (for example for privacy reasons). However, as we have seen, IFs may not work well in the MLP and Deep Learning setting. Therefore, in the literature some algorithms have been developed to unlearn training points without using the highly inefficient LOO retraining. For example, in [10], the authors take inspiration from the *Descend-to-Delete* algorithm (D2D) and create a more efficient version called *Rewind-to-Delete* (R2D).

The former algorithm is very intuitive and consists on simply continuing the training of the model but on the updated dataset where we deleted the target data point. By continuing for enough iteration, there have been proven some theoretical guarantees that the final parameter distribution is in some sense undistinguishable from the one of LOO method.
The problem with this algorithm is that such guarantees only hold in the convex case and cannot be extended to the more general setting (e.g. ReLU and tanh functions are not convex, so this result does not hold).

On the other hand, the R2D algorithm saves a check-point parameter during the training (let's say after the K-th iteration) and then keeps going until iteration T. The idea is that when we receive the request of unlearning the target point, we will continue the training on the updated dataset starting from $\theta(K)$, instead of $\theta(T)$.
The authors of the paper show that also this method yields indistinguishalbility, but in this case the result holds also for non-convex functions and the iterations required are much less than the total training time T (required by LOO).

In [9], the same researchers extend the previous results also to the (projected) SGD case. In particular, in addition to all previous observations, they highlight that D2D provides tighter bounds for the undistinguishability due to its reliance on the convergence to a unique global minimum, while R2D has more loose estimates as it only counts on the underlying contractivity of gradient systems.

Other interesting things we might want to explore

- If our goal is to gain a better understanding of what data points are the most informative during the training, instead of trying to unlearn certain data, the results in [5] might be interesting. In their paper, the authors prove that their Shapley values-based method performs better on this task rather than influence function methods.

- It may be of interest to study Bayesian Influence Functions (BIFs) as well as frequentistic ones. In [7], the researchers present an unlearning method that uses BIFs instead of IFs. The reason for this is we don't need to compute the iHVP to evaluate BIFs, therefore this method works better with more singular loss landscapes. As another consequence, we don't need to evaluate these quantities on local minima (we dont need the hessian to be invertible), which is one of the less realistic hypotheses for IFs.

On the other hand, computations are not always faster than IFs (here, the leading cost is estimating the covariance between two elements) and achieving good results requires more hyperparameter tuning than classical methods.

- Paper about machine unlearning with $\varepsilon \neq 0$ [12]: they introduce a soft-weigthed framework for unlearning data with the aim of improving robustness or fairness. The main point is that if we are not in a privacy-related field, there is no need to completely forget a datapoint; instead, it would be more beneficial to consider it with a lower weight. Moreover, instead of training from scratch the model with the updated weights, the authors propose to correct the result of the usual training with influence functions.

- Something I had not seen yet in machine unlearning, but I have seen for example in plateau explanation [2]: it can make more sense to consider influence functions in certain epochs of the training, not only at the end [8]. Indeed, the importance of a certain data point may change throughout the training.

- One of the reasons why we are analyzing unlearning is that it's kind of the dual problem of finiding influencial samples. However, the authors of [11] state that in general this correlation is not high. Therefore, they propose a new training method for neural nets that increases this alignment while keeping about the same performances of the usual algorithms in terms of training time and accuracy.

Bibliography

[1] Eric Aubinais, Elisabeth Gassiat, and Pablo Piantanida. Fundamental Limits of Membership Inference Attacks on Machine Learning Models, October 2025.

[2] Raphaël Berthier, Andrea Montanari, and Kangjie Zhou. Learning time-scales in two-layers neural networks. *Foundations of Computational Mathematics*, 25(5):1627–1710, August 2024.

[3] Lucas Bourtoule, Varun Chandrasekaran, Christopher A. Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. Machine Unlearning, December 2020. arXiv:1912.03817 [cs].

[4] Olivier Chapelle, Jason Weston, Léon Bottou, and Vladimir Vapnik. Vicinal Risk Minimization. In *Advances in Neural Information Processing Systems*, volume 13. MIT Press, 2000.

[5] Amirata Ghorbani and James Zou. Data Shapley: Equitable Valuation of Data for Machine Learning, June 2019. arXiv:1904.02868 [stat].

[6] Chuan Guo, Tom Goldstein, Awni Hannun, and Laurens van der Maaten. Certified Data Removal from Machine Learning Models, November 2023.

[7] Philipp Alexander Kreer, Wilson Wu, Maxwell Adam, Zach Furman, and Jesse Hoogland. Bayesian Influence Functions for Hessian-Free Data Attribution, September 2025. arXiv:2509.26544 [cs].

[8] Jin Hwa Lee, Matthew Smith, Maxwell Adam, and Jesse Hoogland. Influence Dynamics and Stagewise Data Attribution, October 2025. arXiv:2510.12071 [cs].

[9] Siqiao Mu and Diego Klabjan. Descend or Rewind? Stochastic Gradient Descent Unlearning, November 2025. arXiv:2511.15983 [cs].

[10] Siqiao Mu and Diego Klabjan. Rewind-to-Delete: Certified Machine Unlearning for Nonconvex Functions, October 2025. arXiv:2409.09778 [cs].

[11] Zijie Pan, Zuobin Ying, Yajie Wang, and Wanlei Zhou. On the Misalignment Between Data Learnability and Forgettability in Machine Unlearning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(29), March 2026.

[12] Xinbao Qiao, Ningning Ding, Yushi Cheng, and Meng Zhang. Soft Weighted Machine Unlearning, May 2025. arXiv:2505.18783 [cs].

[13] Maria Refinetti, Alessandro Ingrosso, and Sebastian Goldt. Neural networks trained with SGD learn distributions of increasing complexity. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 28843–28863. PMLR, 23–29 Jul 2023.